

0.22. Remonter : `deplace_y` et un pas négatif

Le 0.21, plus une ligne : `-pas` fait remonter la lettre. Et pourquoi deux fonctions, une par axe.

1. Ce qu'il faut comprendre

C'est le 0.21, plus une ligne : la montée.

Ce qui est nouveau ici : le pas négatif **sur Y**. `-pas` fait **monter** : les numéros de ligne diminuent vers le haut. Comme au 0.19, `-5` est rangé 251, et `deplace_y` lit tout ce qui dépasse 127 comme une montée.

La montée part de `y`, qui vaut 5 grâce au `y =`. Elle rend 0, et `y =` le range.

Pourquoi deux fonctions, et pas une seule pour X et Y ? Une fonction ne peut rendre qu'une seule valeur. `deplace_x` rend la colonne, `deplace_y` rend la ligne : chacune dit exactement où la lettre s'est arrêtée.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1617 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;      // +5 : descend ; -5 : monte
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;        // la LIGNE de la lettre
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    y = deplace_y(0, y, ALPHABET[0], pas);    // la descente (le 0.21) : y vaut 5

    // La ligne ajoutée : le même appel avec -pas, donc vers le HAUT.
    // Elle part de y (5) et rend 0 ; « y = » range ce 0 dans y.
    y = deplace_y(0, y, ALPHABET[0], -pas);    // après cette ligne : y vaut 0

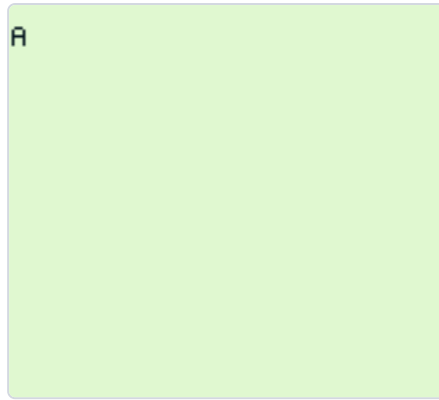
    while (true) {
        image();        // la lettre est revenue tout en haut
    }
}
```

3. Ce qui se passe, étape par étape

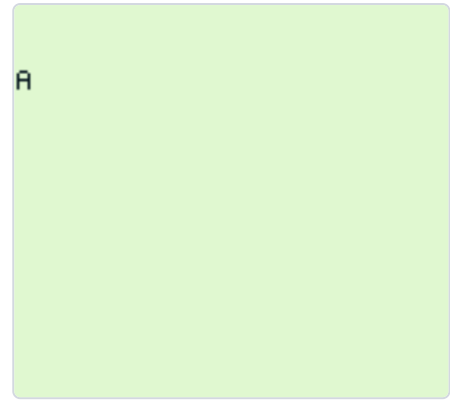
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



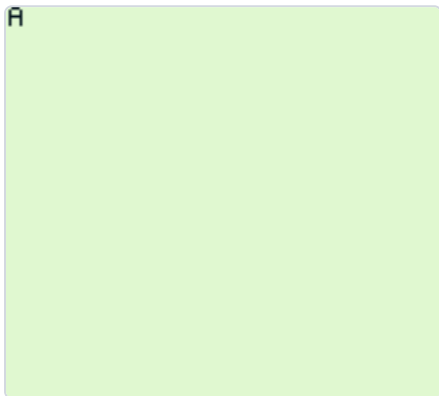
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui descend de 5 lignes, puis remonte de 5 lignes, et s'arrête tout en haut.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ à la descente, le A arrive en ligne 5 (« A »)

✓ à la montée, le A revient en ligne 0 (« A »)

✓ y vaut la ligne rendue ($y = 0$)

Ensuite : **0.22.1. Remonter : `deplace_y` et un pas négatif — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++